

RED TEAM LEARNING PATH

MODULE 01

Linux Fundamentals for Red Teamers

Omarchy / Arch Linux Edition

Fokus modul

- Membangun mental model Linux: kernel, user space, shell, filesystem, process, service, log, dan network.
- Membiasakan command line yang aman dan repeatable pada Omarchy/Arch Linux.
- Memahami user/group, permission, ownership, SUID/SGID/sticky bit, dan file capabilities dari sisi defender dan attacker.
- Menggunakan systemd, journalctl, iproute2, pipes, redirection, grep/find, serta Bash untuk enumerasi lokal yang authorized.
- Menyelesaikan hands-on lab dan mini-project system survey tanpa menyerang sistem pihak lain.

Repository: github.com/VaxXinE/red-teaming

Versi 1.0 - 04 Oktober 2026

Cara Menggunakan Modul Ini

Modul ini sengaja dibuat hands-on. Jangan membaca semua command seperti daftar hafalan. Jalankan latihan pada device Omarchy milik lo sendiri, pahami output-nya, lalu tulis catatan singkat tentang apa yang command tersebut buktikan.

SECURITY NOTE: Semua exercise di modul ini hanya untuk host milik sendiri atau lab yang memang memberi authorization. Perintah enumerasi terlihat “pasif”, tetapi ketika diarahkan ke sistem pihak lain tetap dapat melanggar scope atau kebijakan.

0.1 Learning Objectives

- [] Menjelaskan hubungan kernel, process, shell, filesystem, dan user space.
- [] Menavigasi Linux filesystem dan mengidentifikasi file type, ownership, mode, link, dan metadata.
- [] Membaca serta mengubah permission file milik sendiri secara aman.
- [] Mengobservasi process, signal, service, systemd unit, dan journal.
- [] Membaca network interface, route, DNS state, dan listening socket dari host sendiri.
- [] Menggunakan pipes, redirection, grep, find, sort, uniq, awk/sed secara praktis.
- [] Menulis Bash script read-only yang menghasilkan evidence file dengan permission aman.
- [] Membedakan informasi yang berguna untuk Red Team dengan data sensitif yang tidak boleh dicatat sembarangan.

0.2 Prerequisite

- Module 00 selesai atau minimal sudah memahami scope dan authorized lab.
- Omarchy dapat boot normal dan user mempunyai akses terminal.
- Koneksi internet hanya diperlukan untuk install package/documentation; mayoritas lab berjalan offline.
- Workspace ~/CyberSec tersedia atau akan dibuat pada hands-on pertama.

0.3 Konvensi Command

Notasi	Arti
\$ command	Dijalankan sebagai user biasa.
# command	Membutuhkan privilege root; di Omarchy gunakan sudo command bila memang diperlukan.
<VALUE>	Placeholder yang harus diganti. Jangan mengetik tanda < > secara literal.
~/...	Path relatif terhadap home user saat ini.
read-only	Command mengobservasi state tanpa sengaja mengubah konfigurasi.

COMMON MISTAKE: Jangan biasakan prefix semua command dengan sudo. Privilege harus dinaikkan hanya saat dibutuhkan. Kebiasaan “sudo semuanya” memperbesar blast radius ketika salah path atau salah command.

Daftar Isi

- 1. Mental Model Linux untuk Red Team
- 2. Shell, Command, dan Built-in Help
- 3. Filesystem Hierarchy dan Navigasi
- 4. File Type, Metadata, dan Links
- 5. Users, Groups, Identity, dan Privilege
- 6. Permission, Ownership, dan Special Bits
- 7. Processes, Jobs, dan Signals
- 8. systemd: Services dan Units
- 9. Logs dan Journal
- 10. Linux Networking dari Terminal
- 11. Environment Variables dan PATH
- 12. Pipes, Redirection, dan Text Processing
- 13. Bash Scripting Fundamentals
- 14. Package Management di Omarchy / Arch
- 15. Red Team & Blue Team Perspective
- 16. Hands-on Labs
- 17. Mini Project: Local System Survey
- 18. Knowledge Check
- 19. Practical Challenge
- 20. Cheat Sheet
- 21. Referensi

DESAIN PEMBELAJARAN: Setiap konsep dibawa ke command yang bisa lo jalankan sendiri. Untuk bagian yang punya dampak privilege, modul lebih memilih observasi daripada modifikasi. Tujuannya membangun intuisi yang aman sebelum masuk privilege escalation di modul lanjut.

1. Mental Model Linux untuk Red Team

Sebelum menghafal command, pahami dulu “lapisan” Linux. Saat lo menjalankan sebuah program, shell hanya menjadi pintu masuk. Kernel tetap mengatur CPU, memory, process, filesystem, device, dan networking. Distro seperti Arch mengemas user-space tools; Omarchy menambahkan workflow, package choices, Hyprland, dan konfigurasi di atas Arch.

Mental model sederhana

```

User / Terminal
  |
  v
Shell (bash/zsh/fish)
  |
  v
User-space programs ---> libraries / config / logs
  |
  v
Linux kernel ---> process, memory, filesystem, network, device
  |
  v
Hardware
  
```

Komponen	Fungsi	Kenapa penting buat Red Team
Kernel	Mengatur resource inti OS.	Kernel/version/config memengaruhi attack surface dan observability.
Shell	Menerima command dan melakukan expansion/redirection.	Banyak enumerasi dan automation dimulai dari shell.
Process	Instance program yang sedang berjalan.	Process tree, owner, argumen, dan service dapat memberi konteks penting.
Filesystem	Namespace file, config, device, pseudo-filesystem.	Config, credential artifact, permission, log, dan binary semuanya berada di sini.
systemd	Init dan service manager pada Arch.	Menunjukkan service, startup, failure, timer, dependency, dan log context.
Package manager	Mengelola software dan dependency.	Tooling harus dipasang reproducibly, bukan random curl sh.

RED TEAM PERSPECTIVE: Saat mendapat shell pada lab, pertanyaan awal biasanya bukan “exploit apa?”, tetapi “siapa gue, host apa ini, process/service apa yang hidup, permission apa yang gue punya, interface/route apa yang terlihat, dan artefak apa yang aman untuk diperiksa?”.

BLUE TEAM PERSPECTIVE: Command seperti id, ps, systemctl, find, cat, dan ss menghasilkan process execution. Di endpoint yang dimonitor, urutan command dan parent process dapat menjadi telemetry berguna.

1.1 Hands-on cepat: identitas host

```

whoami
id
hostnamectl
uname -a
cat /etc/os-release
  
```

- [] Catat username dan UID/GID.
- [] Catat kernel release.
- [] Catat bahwa distro base adalah Arch/Omarchy.
- [] Bedakan hostname dengan username.

2. Shell, Command, dan Built-in Help

Shell mengurai command line sebelum program dijalankan. Itu berarti quoting, wildcard, variable expansion, pipeline, dan redirection bisa mengubah arti command. Pemahaman ini penting karena command yang “kelihatan benar” dapat memproses file yang salah kalau quoting-nya buruk.

2.1 Anatomi command

```
command [option] [argument]
```

```
ls -lah /etc
|   |   |
|   |   +-- argument/path
|   +----- options
+----- executable
```

Command	Kegunaan
type -a <cmd>	Menunjukkan bagaimana shell me-resolve command.
command -v <cmd>	Mencari executable/builtin yang akan dipakai.
man <cmd>	Manual page lokal.
<cmd> --help	Quick reference.
apropos <keyword>	Mencari manual page berdasarkan topik.
history	Melihat history shell; perlakukan sebagai data sensitif.

Coba sendiri

```
type -a cd
type -a ls
command -v bash
man ls
ls --help | head
```

SECURITY NOTE: Shell history bisa memuat token, password yang salah ketik, URL signed, atau command sensitif. Jangan masukkan secret ke command line kalau aplikasi menyediakan stdin, file permission-restricted, environment injection yang aman, atau secret manager.

2.2 Quoting dasar

Bentuk	Perilaku
'single quotes'	Literal kuat; variable biasanya tidak diexpand.
"double quotes"	Variable expansion tetap terjadi, tapi spasi di dalam value tetap satu argument.
unquoted	Word splitting dan globbing dapat terjadi; rawan bug pada nama file dengan spasi/wildcard.

Bandingkan output

```
name='lab file.txt'
printf '%s\n' "$name"
printf '%s\n' $name
```

COMMON MISTAKE: Dalam script, biasakan quote variable path: "\$file", "\$dir", "\$output". Ini mencegah word splitting dan banyak bug klasik.

3. Filesystem Hierarchy dan Navigasi

Linux menggunakan satu tree yang dimulai dari /. Directory seperti /etc, /var, /proc, /sys, /run, /tmp, /usr, dan /home punya peran berbeda. man hier(7) dan file-hierarchy(7) mendeskripsikan struktur umum; distro modern dapat memakai symlink atau layout tambahan.

Path	Makna praktis	Perhatian Red Team
/	Root filesystem tree.	Mulai mapping host, mount point, dan

Path	Makna praktis	Perhatian Red Team
		boundary.
/etc	Host-specific configuration.	Service config, auth config, network/system settings.
/home	Home user.	Dotfiles, project files, SSH config; jangan asal menyalin data.
/root	Home user root.	Biasanya tidak dapat diakses user biasa.
/var	Variable state.	Log, cache, state aplikasi, database lokal.
/tmp	Temporary data.	Permission dan cleanup policy penting; jangan anggap "private".
/run	Runtime state sejak boot.	PID/socket/runtime data.
/proc	Pseudo-filesystem process/kernel.	Process metadata, kernel info, mounts, environ dengan permission tertentu.
/sys	Kernel/device model via sysfs.	Device dan kernel object information.
/usr	Binaries, libraries, shared data.	Sebagian besar tool user-space.
/opt	Optional/add-on software.	Kadang aplikasi vendor/custom.

Navigasi awal

```
pwd
ls -lah /
ls -lah /etc | head
ls -lah /var/log | head
mount | head
df -hT
```

3.1 Path absolut vs relatif

```
cd ~/CyberSec
pwd

# relatif
cd labs

# absolut
cd /etc

# kembali ke home
cd ~
```

THINK LIKE AN ATTACKER: Kalau lo melihat path seperti /opt/vendor/app atau /srv/internal, jangan langsung menyimpulkan vulnerable. Pertanyaan yang benar: siapa owner-nya, permission apa, process apa yang memakai file itu, dan apakah lo memang authorized memeriksanya?

4. File Type, Metadata, dan Links

Nama file di Linux tidak menentukan tipenya. Gunakan metadata dan content-aware tools seperti `file(1)`, `stat(1)`, dan `ls -l`. Symlink menunjuk ke path lain, sedangkan hard link menunjuk inode yang sama pada filesystem yang kompatibel.

Prefix ls -l	Jenis
-	Regular file
d	Directory
l	Symbolic link
c	Character device
b	Block device

Prefix ls -l	Jenis
s	Socket
p	FIFO / named pipe

```
mkdir -p ~/CyberSec/labs/module01/links
cd ~/CyberSec/labs/module01/links
printf 'hello
' > original.txt
ln original.txt hardlink.txt
ln -s original.txt symlink.txt

ls -li
file original.txt symlink.txt
stat original.txt
readlink symlink.txt
realpath symlink.txt
```

CONCEPT: Hard link berbagi inode dengan file asal. Symlink memiliki inode sendiri dan menyimpan referensi path. Memahami link penting saat menganalisis permission, deployment, config, dan potensi path confusion.

SECURITY NOTE: Jangan menghapus atau memodifikasi file sistem hanya karena link terlihat “aneh”. Pada host nyata, symlink dapat merupakan bagian normal packaging/configuration.

5. Users, Groups, Identity, dan Privilege

Linux mengaitkan access control ke UID/GID. Username hanyalah representasi manusia dari identifier tersebut. Sebuah process berjalan dengan identity dan group context tertentu; itulah alasan output id sangat penting saat melakukan assessment.

```
whoami
id
groups
getent passwd "$USER"
getent group | head
```

Konsep	Penjelasan
UID	Numeric user identifier. UID 0 adalah root.
GID	Primary group identifier.
Supplementary groups	Group tambahan yang memberi akses ke resource tertentu.
root	Superuser dengan privilege sangat luas.
wheel	Group yang sering dipakai untuk administrative privilege via sudo policy pada Arch.
service account	Identity untuk daemon/service; seharusnya memakai least privilege.

5.1 sudo bukan mode kerja default

sudo menjalankan command dengan identity yang lebih tinggi sesuai policy. Gunakan hanya untuk command yang memang memerlukan privilege. Perhatikan command lengkap sebelum Enter, terutama rm, chmod/chown recursive, package manager, mount, dan file di /etc.

```
sudo -l

# lihat identity ketika command memang dijalankan via sudo
sudo id
```

SECURITY NOTE: Output sudo -l dapat mengandung informasi sensitif tentang privilege policy. Simpan sebagai evidence hanya pada lab/engagement authorized dan jangan publish mentah jika ada username/path internal.

6. Permission, Ownership, dan Special Bits

Mode klasik Linux membagi permission menjadi owner, group, dan others; masing-masing punya read (r=4), write (w=2), execute (x=1). Untuk directory, execute berarti traverse/search, bukan sekadar “menjalankan folder”.

Cara membaca mode

```
ls -l secret.txt
# -rw----- 1 user user ... secret.txt
#  |  |  |  |  |  |
#  u   g   o
```

Mode	Makna file	Makna directory
r	Baca content.	List nama entry jika juga diizinkan oleh directory semantics.
w	Ubah content.	Buat/hapus/rename entry tergantung permission directory.
x	Execute binary/script.	Traverse directory.

6.1 chmod symbolic dan numeric

```
cd ~/CyberSec/labs/module01
printf 'training-only
' > secret.txt
chmod 600 secret.txt
ls -l secret.txt
```

```
chmod u+x system-info.sh
chmod 750 system-info.sh
```

Octal	Umum digunakan untuk
600	Private file: owner read/write.
640	Owner read/write, group read.
700	Private directory/script.
750	Owner full, group read/execute.
755	Executable/public-readable directory tree.

6.2 umask

umask memengaruhi permission awal file/directory baru. Untuk evidence atau data lab sensitif, umask 077 membantu agar file baru hanya dapat diakses owner.

```
umask
umask 077
mkdir private-evidence
printf 'local lab evidence
' > private-evidence/note.txt
ls -ld private-evidence
ls -l private-evidence/note.txt
```

SECURITY NOTE: umask bukan pengganti encryption. Permission Linux melindungi saat OS sedang berjalan; data-at-rest encryption diperlukan untuk risiko physical access.

6.3 SUID, SGID, sticky bit, capabilities

Special bits dan capabilities memungkinkan pola privilege yang lebih granular. Di tahap ini kita hanya mengobservasi; exploitation dibahas pada module privilege escalation nanti.


```
# read-only observation
find /usr/bin -perm -4000 -type f 2>/dev/null
find /usr/bin -perm -2000 -type f 2>/dev/null

# jika getcap tersedia
getcap -r /usr/bin 2>/dev/null | head
```

RED TEAM PERSPECTIVE: SUID/SGID/capabilities menarik karena executable dapat berjalan dengan privilege atau capability berbeda dari user yang memanggilnya. Namun keberadaan bit khusus tidak otomatis berarti vulnerability.

7. Processes, Jobs, dan Signals

Process memiliki PID, parent PID, user, executable, environment, open files, sockets, dan resource usage. /proc adalah pseudo-filesystem yang mengekspos banyak metadata process sesuai permission.

```
ps aux | head
ps -eo pid,ppid,user,stat,comm,args --sort=pid | head
pgrep -a bash
cat /proc/$$/status | head
```

7.1 Job control

```
sleep 300 &
jobs -l
pgrep -a sleep
kill -TERM <PID>
jobs -l
```

SIGTERM meminta process berhenti secara normal. SIGKILL memaksa kernel menghentikan process dan tidak dapat ditangani process; jangan menjadikan kill -9 sebagai kebiasaan pertama.

Signal	Makna praktis
SIGTERM (15)	Graceful termination request.
SIGINT (2)	Interrupt, biasanya dari Ctrl+C.
SIGHUP (1)	Historis hangup; banyak daemon menggunakannya untuk reload.
SIGKILL (9)	Force kill; tidak bisa di-catch/ignore.

7.2 Open files dan sockets

```
lsof -p $$ | head
ss -ltnup
```

BLUE TEAM PERSPECTIVE: Process ancestry, command line, executable path, user, open network socket, dan timing merupakan telemetry penting. Sequence enumerasi yang sangat cepat dapat terlihat berbeda dari aktivitas admin normal.

8. systemd: Services dan Units

Arch Linux menggunakan systemd sebagai init dan service manager. Unit dapat berupa service, socket, timer, mount, target, dan tipe lain. Untuk belajar Red Team, fokus awal: apa yang running, enabled, failed, dan siapa yang menjalankannya.

```
systemctl --type=service --state=running
systemctl --failed
systemctl list-unit-files --type=service | head
systemctl status systemd-journald --no-pager
```

8.1 system vs user units

```
systemctl --user --type=service --state=running
systemctl --user --failed
```

Command	Pertanyaan yang dijawab
systemctl status X	Apa unit aktif, PID utama, recent log, dan status exit?
systemctl cat X	Unit file/drop-in apa yang membentuk service?
systemctl show X	Property machine-readable apa yang dimiliki unit?
systemctl --failed	Adakah service yang gagal?
systemctl list-timers	Timer apa yang dijadwalkan?

SECURITY NOTE: Jangan enable/disable/restart service produksi hanya untuk “mencoba”. Pada module ini gunakan observation commands. Perubahan state service termasuk aksi yang harus sesuai scope.

9. Logs dan Journal

systemd journal mengumpulkan event dari service, kernel, dan user units. Pada Arch, journalctl adalah tool utama untuk membaca journal. Akses regular user dapat terbatas; jangan naikan privilege hanya untuk membaca data yang tidak perlu.

```
journalctl -b -n 30 --no-pager
journalctl -b -p warning --no-pager
journalctl --user -b -n 30 --no-pager
journalctl -u systemd-journald -n 20 --no-pager
```

9.1 Filtering yang lebih baik

```
journalctl --since '20 min ago' --no-pager
journalctl --grep='error' -n 20 --no-pager
journalctl -b -k -n 30 --no-pager
```

BLUE TEAM PERSPECTIVE: Journal membantu membangun timeline. Di engagement yang authorized, catat timestamp dan timezone evidence. Jangan “membersihkan log” setelah lab kecuali challenge secara eksplisit meminta dan menyediakan disposable environment.

COMMON MISTAKE: journalctl | grep memang bisa bekerja, tetapi journalctl punya filter native seperti --unit, --priority, --since, dan --grep yang lebih efisien serta ekspresif.

10. Linux Networking dari Terminal

Di Arch, iproute2 menyediakan command ip untuk interface, address, dan routing. ifconfig/netstat adalah legacy pada banyak tutorial. Di modul ini gunakan ip dan ss sebagai default modern.

```
ip -brief address
ip route
ip neigh
ss -lntup
resolvectl status
```

Command	Informasi
ip -brief address	Interface dan IP address ringkas.
ip route	Routing table dan default gateway.
ip neigh	Neighbor/ARP/NDP cache.

Command	Informasi
ss -lntup	Listening TCP/UDP sockets dan process info jika diizinkan.
resolvectl status	Resolver/DNS state pada systemd-resolved setup.

10.1 Loopback dan exposure

Service yang bind ke 127.0.0.1 hanya menerima traffic lokal IPv4 pada interface loopback. Service yang bind ke 0.0.0.0 menerima pada semua IPv4 interface yang relevan, subject to firewall/network policy. Ini alasan kita sering publish Docker lab ke 127.0.0.1 pada Module 00.

```
ss -lnt

# contoh pola yang aman untuk local lab
# 127.0.0.1:8080 -> localhost only
# 0.0.0.0:8080 -> semua IPv4 interface yang cocok
```

SECURITY NOTE: Jangan mengubah firewall, route, atau bind address host utama tanpa alasan jelas. Untuk training service, localhost-only exposure adalah default yang baik.

11. Environment Variables dan PATH

Environment adalah pasangan NAME=value yang diwariskan ke child process. PATH menentukan directory yang dicari shell untuk executable. Environment sering memuat konfigurasi runtime dan kadang secret, jadi jangan dump semuanya ke laporan atau Git.

```
printenv PATH
printf '%s\n' "$SHELL"
printf '%s\n' "$HOME"
command -v ls
type -a ls
```

11.1 Temporary environment

```
LAB_MODE=training bash -c 'printf "%s\n" "$LAB_MODE"'

export LAB_NAME='module01'
printf '%s\n' "$LAB_NAME"
unset LAB_NAME
```

SECURITY NOTE: Environment variables bukan secret manager. Pada banyak runtime, environment process dapat terlihat oleh process lain dengan permission yang cukup dan dapat masuk crash dump/log/debug output.

11.2 PATH hygiene

```
printf '%s\n' "$PATH" | tr ':' ' '
command -v python
command -v nmap || true
```

Hindari menaruh current directory (.) di depan PATH secara sembarangan. Jika sebuah directory writable oleh pihak yang tidak dipercaya muncul lebih awal di PATH, executable bernama sama dapat dipilih lebih dulu.

12. Pipes, Redirection, dan Text Processing

Pipeline menghubungkan stdout command sebelumnya ke stdin command berikutnya. Redirection mengatur stdin/stdout/stderr. Ini adalah “lem” utama automation Linux dan sangat penting untuk parsing hasil enumerasi.

Syntax	Makna
cmd1 cmd2	stdout cmd1 menjadi stdin cmd2.
> file	Overwrite stdout ke file.
>> file	Append stdout ke file.
2> file	Redirect stderr.
2>&1	Gabungkan stderr ke destination stdout.
< file	Gunakan file sebagai stdin.
tee file	Tulis ke file sambil tetap tampil ke stdout.

Pipeline sederhana

```
printf '%s\n' alpha beta alpha gamma | sort | uniq -c | sort -nr
```

12.1 grep, find, awk, sed, cut, sort, uniq

```
grep -n 'root' /etc/passwd
getent passwd | cut -d: -f1 | sort | head
find ~/CyberSec/labs/module01 -type f -name '*.txt' -print

printf 'user:1000\nroot:0\n' | awk -F: '{print $1, $2}'
```

12.2 Safer filename handling

Filename bisa mengandung spasi, newline, wildcard, dan karakter lain. Untuk pipeline file, gunakan null-delimited output ketika tool mendukungnya.

```
find ~/CyberSec/labs/module01 -type f -print0 | xargs -0 -r ls -l
```

COMMON MISTAKE: Jangan parse output ls untuk automation. Gunakan find -print0, stat, atau API/tool yang memang machine-readable.

13. Bash Scripting Fundamentals

Tujuan scripting di fase ini bukan membuat malware atau exploit. Kita belajar mengubah langkah manual menjadi procedure yang konsisten, aman, mudah direview, dan menghasilkan evidence yang jelas.

Skeleton yang kita pakai

```
#!/usr/bin/env bash
set -euo pipefail

main() {
    printf 'hello from module01\n'
}

main "$@"
```

Praktik	Alasan
#!/usr/bin/env bash	Cari Bash melalui environment PATH.
set -euo pipefail	Fail lebih cepat pada banyak error; bukan security boundary.
quote "\$var"	Hindari word splitting/globbering tidak disengaja.
printf	Lebih predictable dibanding echo untuk banyak kasus.
functions	Memisahkan concern dan membuat script lebih mudah dibaca.

Praktik	Alasan
read-only by default	Kurangi risiko merusak host saat enumerasi.

13.1 Input validation mini

```
#!/usr/bin/env bash
set -euo pipefail

output_dir=${1:-}

if [[ -z "$output_dir" ]]; then
    printf 'usage: %s OUTPUT_DIR\n' "$0" >&2
    exit 2
fi

mkdir -p -- "$output_dir"
printf 'created: %s\n' "$output_dir"
```

SECURITY NOTE: Jangan menggunakan eval pada input yang tidak dipercaya. Jangan membuat command string lalu mengeksekusinya kembali jika array atau argument langsung sudah cukup.

14. Package Management di Omarchy / Arch

Omarchy menyediakan wrapper untuk install/remove package dan update sistem. Untuk paket normal, gunakan menu Install > Package atau CLI omarchy pkg add. Untuk system update, Omarchy merekomendasikan omarchy update agar snapshot, migration, config update, dan package update berjalan bersama.

```
omarchy pkg add tree lsof ripgrep
omarchy pkg drop tree
omarchy update
```

SECURITY NOTE: AUR tidak divalidasi oleh Arch team seperti official repositories. Review PKGBUILD, upstream URL, source checksum, dan install script sebelum memasang AUR package. Hindari random installer curl | sh jika ada package/reproducible method yang lebih baik.

14.1 Toolset Module 01

Tool/package	Status	Dipakai untuk
bash	base	Shell dan scripting.
coreutils	base	ls, cat, cp, mv, chmod, chown, sort, uniq, dll.
procs-ng	base/umum	ps dan process utilities.
iproute2	base	ip dan ss.
systemd	base	systemctl, journalctl.
tree	opsional	Visualisasi directory tree.
lsof	opsional	Open files/socket inspection.
ripgrep	opsional	Fast recursive text search.
libcap	opsional	getcap/setcap utilities.

Optional

```
# install hanya yang belum ada
omarchy pkg add tree lsof ripgrep libcap
```

15. Red Team & Blue Team Perspective

15.1 Red Team enumeration mindset

```

IDENTITY
-> who am I?
HOST
-> OS / kernel / hostname?
FILESYSTEM
-> interesting paths / mounts / permissions?
PROCESS
-> what is running, under which identity?
SERVICE
-> what starts automatically and how?
NETWORK
-> interfaces, routes, listeners?
LOGS
-> what happened and what will defenders see?
PRIVILEGE
-> what can this identity legitimately do?

```

Urutan itu bukan checklist kaku. Tujuannya membangun model host secara bertahap dan memilih command paling sedikit yang cukup untuk menjawab pertanyaan.

15.2 Blue Team observability mindset

Aktivitas	Telemetry potensial
Menjalankan shell command	Process creation, shell history, terminal/session logs tergantung setup.
Membaca banyak file	Endpoint/audit telemetry tergantung policy.
Membuka socket/listener	Network/process telemetry.
sudo	Authentication/authorization log dan journal.
systemctl	Process execution dan kemungkinan unit state change.
Package install	Package database/log, filesystem change, network download.

THINK LIKE AN ATTACKER: Operator yang matang tidak hanya bertanya “bisa dilakukan?”, tetapi juga “apakah perlu dilakukan, apakah sesuai scope, evidence apa yang dihasilkan, dan telemetry apa yang kemungkinan muncul?”.

16. Hands-on Labs

Semua lab berikut dijalankan pada Omarchy milik lo sendiri. Sebelum mulai, buat workspace dan pastikan evidence baru tidak world-readable.

```
mkdir -p ~/CyberSec/labs/module01/{work,evidence,scripts}
chmod 700 ~/CyberSec/labs/module01/evidence
cd ~/CyberSec/labs/module01
```

16.1 Lab A - Host Identity Baseline

HANDS-ON: Membuat baseline host sendiri tanpa mengubah konfigurasi.

```
{
  printf '== identity ==\n'
  whoami
  id
  printf '\n== host ==\n'
  hostnamectl
  printf '\n== kernel ==\n'
  uname -a
  printf '\n== os ==\n'
  cat /etc/os-release
} | tee evidence/identity.txt
```

- [] File evidence/identity.txt terbuat.
- [] Lo dapat menjelaskan UID, GID, hostname, kernel, dan distro.
- [] Tidak ada secret/token yang masuk ke evidence.

16.2 Lab B - Filesystem Scavenger

HANDS-ON: Melatih navigation, metadata, link, dan file search hanya di workspace sendiri.

```
cd ~/CyberSec/labs/module01/work
mkdir -p alpha/beta gamma
printf 'one\n' > alpha/one.txt
printf 'two\n' > alpha/beta/two.txt
ln -s alpha/one.txt one-link.txt

tree .
find . -maxdepth 3 -type f -print
stat alpha/one.txt
file alpha/one.txt one-link.txt
readlink one-link.txt
realpath one-link.txt
```

- [] Bedakan regular file dan symlink.
- [] Temukan inode/metadata dengan stat.
- [] Jelaskan perbedaan relative symlink target dan resolved path.

16.3 Lab C - Permission Challenge

HANDS-ON: Membentuk intuition rwx, octal mode, dan umask tanpa menyentuh file sistem.

```
cd ~/CyberSec/labs/module01/work
umask 077
printf 'private\n' > private.txt
mkdir private-dir

ls -ld private.txt private-dir
chmod 640 private.txt
chmod 750 private-dir
ls -ld private.txt private-dir
stat -c '%A %a %U:%G %n' private.txt private-dir
```

Jawab di notes: mengapa directory 640 sering tidak berguna untuk akses normal? Apa fungsi execute pada directory?

CHECKPOINT: Target: lo bisa menerjemahkan rwxr-x--- menjadi 750 tanpa kalkulator dan menjelaskan implikasinya untuk file versus directory.

16.4 Lab D - Process & Signal

HANDS-ON: Mengobservasi process lifecycle dan menghentikan process milik sendiri secara graceful.

```
sleep 300 &
PID=$!
printf 'pid=%s\n' "$PID"
ps -o pid,ppid,user,stat,comm,args -p "$PID"
cat "/proc/$PID/status" | head
kill -TERM "$PID"
wait "$PID" 2>/dev/null || true
```

- [] Identifikasi PID dan PPID.
- [] Process hilang setelah SIGTERM.
- [] Tidak menggunakan kill -9.

16.5 Lab E - Service & Journal Observation

HANDS-ON: Menghubungkan unit systemd dengan recent log tanpa mengubah service state.

```
systemctl status systemd-journald --no-pager
systemctl --failed --no-pager
journalctl -u systemd-journald -n 20 --no-pager
journalctl -b -p warning -n 20 --no-pager
```

Kalau output tertentu membutuhkan permission yang tidak lo punya, catat "permission denied" sebagai bagian observasi. Jangan otomatis sudo hanya agar output penuh.

16.6 Lab F - Network Baseline

HANDS-ON: Membaca interface, route, DNS state, dan listener host sendiri.

```
{
  printf '== address ==\n'
  ip -brief address
  printf '\n== route ==\n'
  ip route
  printf '\n== listeners ==\n'
  ss -ltnup
  printf '\n== dns ==\n'
  resolvectl status 2>/dev/null || true
} | tee evidence/network.txt
```

- [] Temukan interface loopback.
- [] Temukan default route jika ada.
- [] Bedakan LISTEN socket localhost-only dan wildcard bind.

16.7 Lab G - Pipeline Gym

HANDS-ON: Mengubah raw text menjadi informasi ringkas menggunakan pipeline.

```
cat > work/sample.log <<'EOF'
INFO login alice
WARN retry bob
INFO logout alice
ERROR denied charlie
WARN retry bob
INFO login bob
```


EOF

```
awk '{print $1}' work/sample.log | sort | uniq -c | sort -nr
grep -n '^WARN' work/sample.log
awk '{print $3}' work/sample.log | sort | uniq -c | sort -nr
```

Challenge: buat satu pipeline yang hanya menampilkan username yang muncul minimal dua kali. Jangan gunakan Python dulu.

16.8 Lab H - Read-only Special Permission Survey

HANDS-ON: Mengobservasi SUID/SGID/capability tanpa mengeksploitasi apa pun.

```
find /usr/bin -perm -4000 -type f 2>/dev/null | sort | tee evidence/suid.txt
find /usr/bin -perm -2000 -type f 2>/dev/null | sort | tee evidence/sgid.txt
getcap -r /usr/bin 2>/dev/null | sort | tee evidence/capabilities.txt
```

SECURITY NOTE: Exercise berhenti pada inventory. Jangan mencoba abuse binary hanya karena memiliki SUID/capability. Analisis exploitability dibahas di Linux Privilege Escalation dengan lab yang sengaja vulnerable.

16.9 Lab I - Safer Search

HANDS-ON: Berlatih menemukan file berdasarkan nama/content di workspace tanpa parsing ls.

```
find ~/CyberSec/labs/module01/work -type f -name '*.txt' -print0 | xargs -0 -r grep -nH 'alice' || true
rg -n 'alice' ~/CyberSec/labs/module01/work 2>/dev/null || true
```

Bandungkan output find+xargs+grep dengan ripgrep. Tulis kapan tool generic lebih portable dan kapan rg lebih nyaman.

17. Mini Project: Local System Survey

Sekarang gabungkan konsep module menjadi script read-only. Script ini hanya menginventarisasi host lokal dan menulis hasil ke evidence file dengan permission ketat. Jangan menambahkan dump environment penuh, browser data, SSH private key, atau secret lain.

scripts/system-survey.sh

```
#!/usr/bin/env bash
set -euo pipefail

OUT_DIR=${1:-"$HOME/CyberSec/labs/module01/evidence"}
mkdir -p -- "$OUT_DIR"
chmod 700 -- "$OUT_DIR"
umask 077

STAMP=$(date -u +%Y%m%dT%H%M%S)
OUT="$OUT_DIR/system-survey-$STAMP.txt"

section() {
    printf '\n==== %s =====\n' "$1"
}

{
    section 'TIME'
    date --iso-8601=seconds

    section 'IDENTITY'
    whoami
    id

    section 'HOST'
    hostnamectl 2>/dev/null || hostname
    uname -a
    printf '\n'
```

```

cat /etc/os-release 2>/dev/null || true

section 'FILESYSTEM'
df -hT
printf '\nMounts (first 30):\n'
findmnt | head -n 30

section 'PROCESS SNAPSHOT'
ps -eo pid,ppid,user,stat,comm,args --sort=pid | head -n 40

section 'FAILED SERVICES'
systemctl --failed --no-pager 2>/dev/null || true

section 'NETWORK'
ip -brief address
printf '\nRoutes:\n'
ip route
printf '\nListening sockets:\n'
ss -lntup 2>/dev/null || ss -lntu

section 'RECENT WARNINGS'
journalctl -b -p warning -n 30 --no-pager 2>/dev/null || true
} > "$OUT"

chmod 600 -- "$OUT"
printf 'survey written to: %s\n' "$OUT"

```

Cara menjalankan

```

cd ~/CyberSec/labs/module01
cat > scripts/system-survey.sh <<'EOF'
# paste script dari modul di sini
EOF
chmod 700 scripts/system-survey.sh
./scripts/system-survey.sh
ls -l evidence/system-survey-*.txt

```

SECURITY NOTE: Sebelum commit repo publik, review evidence. Output hostname, username, IP internal, process args, mount, dan log bisa sensitif. Default repository kita sebaiknya menyimpan script/template, bukan raw host evidence.

17.1 Review script seperti engineer

- [] Script memakai set -euo pipefail namun command opsional tetap diberi fallback yang disengaja.
- [] Semua variable path di-quote.
- [] Output directory permission 700 dan evidence 600.
- [] Tidak memakai eval.
- [] Tidak melakukan package install, service restart, network scan, atau perubahan konfigurasi.
- [] Tidak dump environment penuh atau credential material.
- [] Output memiliki timestamp UTC untuk membantu timeline.

CHECKPOINT: Kalau lo mampu menjelaskan setiap command pada script tanpa sekadar membacanya, fondasi shell/Linux lo sudah cukup untuk lanjut ke Networking Fundamentals.

18. Knowledge Check

1. Apa perbedaan kernel, shell, dan distro?

Jawaban: _____

2. Apa beda path absolut dan path relatif?

Jawaban: _____

3. Apa makna d, -, dan l pada karakter pertama output ls -l?

Jawaban: _____

4. Apa perbedaan hard link dan symbolic link?

Jawaban: _____

5. Apa yang ditunjukkan UID/GID pada command id?

Jawaban: _____

6. Terjemahkan rwxr-x--- ke octal dan jelaskan maknanya untuk directory.

Jawaban: _____

7. Mengapa execute bit pada directory berbeda dari execute bit pada regular file?

Jawaban: _____

8. Mengapa SUID binary tidak otomatis berarti vulnerability?

Jawaban: _____

9. Apa beda SIGTERM dan SIGKILL?

Jawaban: _____

10. Apa fungsi systemctl --failed?

Jawaban: _____

11. Apa keuntungan filter native journalctl dibanding journalctl | grep?

Jawaban: _____

12. Mengapa ip/ss lebih disukai dibanding tutorial lama yang memakai ifconfig/netstat?

Jawaban: _____

13. Apa risiko memasukkan . di depan PATH?

Jawaban: _____

14. Apa beda > dan >>?

Jawaban: _____

15. Mengapa find -print0 + xargs -0 lebih aman untuk filename aneh?

Jawaban: _____

16. Mengapa environment variable tidak boleh dianggap secret manager?

Jawaban: _____

17. Kenapa raw evidence system survey tidak seharusnya masuk repo publik?

Jawaban: _____

18. Kapan sudo seharusnya digunakan?

Jawaban: _____

19. Apa yang dilakukan umask 077 secara konsep?

Jawaban: _____

20. Mengapa operator Red Team perlu memahami telemetry Blue Team?

Jawaban: _____

19. Practical Challenge - 30 Menit

Tanpa melihat cheat sheet selama 30 menit, hasilkan evidence lokal yang menjawab semua objective di bawah. Gunakan hanya host Omarchy sendiri.

[] Tulis username, UID/GID, hostname, distro, dan kernel ke evidence/baseline.txt.

[] Buat work/challenge.txt dengan mode 600 dan buktikan dengan stat.

[] Buat symlink work/challenge-link -> challenge.txt dan tunjukkan target/resolved path.

[] Jalankan sleep di background, temukan PID/PPID, lalu hentikan dengan SIGTERM.

- [] Tampilkan failed systemd services tanpa mengubah state service.
- [] Tampilkan 10 warning terbaru dari current boot jika diizinkan.
- [] Tulis interface, route, dan listening socket ke evidence/network-challenge.txt.
- [] Gunakan pipeline untuk menghitung jumlah user dari getent passwd tanpa hard-code angka.
- [] Inventory SUID binary /usr/bin ke evidence/suid-challenge.txt; jangan exploit.
- [] Tulis script challenge-summary.sh yang mencetak lokasi ketiga evidence file dan permission-nya.

Kriteria	Bobot
Correctness: output menjawab objective.	40%
Safety: tidak mengubah system config/service/network.	25%
Evidence hygiene: permission dan path rapi.	20%
Clarity: command/script dapat dibaca dan dijelaskan.	15%

PASS CRITERIA: Target minimal 80%. Kalau masih bergantung copy-paste tanpa memahami output, ulangi lab A-G sebelum lanjut Module 02.

20. Cheat Sheet

Topik	Command cepat
Identity	whoami ; id ; groups
OS/kernel	hostnamectl ; uname -a ; cat /etc/os-release
Navigation	pwd ; cd ; ls -lah ; realpath PATH
Metadata	file PATH ; stat PATH ; ls -li
Links	readlink LINK ; realpath LINK
Permissions	chmod 600 FILE ; chmod 700 DIR ; stat -c "%A %a %U:%G %n" PATH
Users/groups	getent passwd ; getent group ; sudo -l
Process	ps aux ; pgrep -a NAME ; cat /proc/PID/status
Jobs/signals	jobs -l ; kill -TERM PID
Services	systemctl --type=service --state=running ; systemctl --failed
Journal	journalctl -b -n 30 ; journalctl -u UNIT ; journalctl --since "20 min ago"
Network	ip -brief address ; ip route ; ss -ltnup
DNS	resolvectl status
PATH	printenv PATH ; command -v NAME ; type -a NAME
Search	find PATH ... ; grep -n PATTERN FILE ; rg PATTERN PATH
Pipeline	sort ; uniq -c ; cut ; awk ; sed ; tee
Package	omarchy pkg add NAME ; omarchy pkg drop NAME
System update	omarchy update

NEXT MODULE: MODULE 02 - Networking Fundamentals. Kita akan membedah TCP/IP, subnetting, routing, ARP, DNS, TCP/UDP, ports, Wireshark/tcpdump, dan membangun lab network lokal supaya output ip/ss di module ini benar-benar punya makna.

21. Referensi

Referensi berikut dipakai sebagai anchor untuk perilaku Omarchy/Arch/Linux yang dibahas pada modul. Akses terakhir: 04 Oktober 2026.

[R1] Omarchy Manual - Other Packages - <https://omarchy.org/manual/other-packages/> - omarchy pkg add/drop dan peringatan AUR.

[R2] Omarchy Manual - Updates - <https://omarchy.org/manual/updates/> - omarchy update, snapshot/migration/config update, dan warning direct system upgrade.

[R3] Omarchy Manual - CLI - <https://omarchy.org/manual/omarchy-cli/> - command center dan CLI Omarchy.

[R4] Omarchy Manual - Dotfiles - <https://omarchy.org/manual/dotfiles/> - konfigurasi user di ~/.config dan pemisahan dari internal /usr/share/omarchy.

[R5] ArchWiki - General recommendations - https://wiki.archlinux.org/title/General_recommendations - systemd/service management dan maintenance orientation.

[R6] ArchWiki - File permissions and attributes - https://wiki.archlinux.org/title/File_permissions_and_attributes - rwx, ownership, ACL, umask, attributes.

[R7] ArchWiki - systemd/Journal - <https://wiki.archlinux.org/title/Systemd/Journal> - journalctl access, filtering, persistent journal.

[R8] ArchWiki - Network configuration / iproute2 - https://wiki.archlinux.org/title/Network_configuration - iproute2 dan modern network CLI.

[R9] ArchWiki - System maintenance - https://wiki.archlinux.org/title/System_maintenance - systemctl --failed dan journal checks.

[R10] Linux man-pages - hier(7) - <https://man7.org/linux/man-pages/man7/hier.7.html> - filesystem hierarchy.

[R11] Linux man-pages - proc(5) - <https://man7.org/linux/man-pages/man5/proc.5.html> - /proc sebagai process/kernel pseudo-filesystem.

[R12] Linux man-pages - environ(7) - <https://man7.org/linux/man-pages/man7/envron.7.html> - environment model dan inheritance.

[R13] Linux man-pages - proc_pid_environ(5) - https://man7.org/linux/man-pages/man5/proc_pid_environ.5.html - process initial environment dan access control.

[R14] GNU Bash Reference Manual - <https://www.gnu.org/software/bash/manual/bash.html> - pipelines, redirection, expansion, shell behavior.

[R15] systemd file-hierarchy(7) - <https://man7.org/linux/man-pages/man7/file-hierarchy.7.html> - modern systemd filesystem hierarchy context.

Appendix A - Suggested Notes Template

```
# Module 01 - Linux Fundamentals
## Concept yang masih belum jelas
-
## Command baru
- command: | tujuan: | output penting:
## Hands-on result
- Lab A: | Lab B: | Lab C:
## Mistake / surprise
-
## Red Team insight
-
## Blue Team insight
-
## Pertanyaan untuk review
-
```

Appendix B - Lab Cleanup

Cleanup hanya menghapus workspace training Module 01. Review dulu evidence yang mau dipertahankan.

```
cd ~/CyberSec/labs

# review dulu
find module01 -maxdepth 2 -type f -print

# hapus hanya jika yakin
rm -rf -- module01
```

END OF MODULE: MODULE 01 complete. Jangan lanjut sebelum practical challenge bisa diselesaikan tanpa copy-paste buta dan lo dapat menjelaskan permission, process, service, log, network state, serta pipeline dasar dengan bahasa sendiri.